

INTERNSHIP REPORT

on

STUDENT RESULT PREDICTION USING MACHINE LEARNING

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Technology In
Computer Science & Engineering (Data Science)



Government Engineering College, Lakhisarai

Under the supervision of



Riya Kumari

(UpEducative Technology Private Limited)

Name	Isha
Registration No.	25153158029
Branch	Computer Science & Engineering (Data Science)
Session	2026–2027
College	Government Engineering College, Lakhisarai

CERTIFICATE

This is to certify that Isha, a student of B.Tech (Computer Science & Engineering – Data Science) at Government Engineering College, Lakhisarai, has satisfactorily completed an internship training programme titled “Student Result Prediction using Machine Learning”, conducted under the training partnership of Upeducative, during the academic session 2026–2027.

During this internship, the student was engaged in a structured, project-based curriculum covering numerical computing with NumPy, data handling with Pandas, data cleaning, exploratory data analysis, feature scaling, model building, and model evaluation, culminating in a working machine learning pipeline for predicting student academic outcomes.

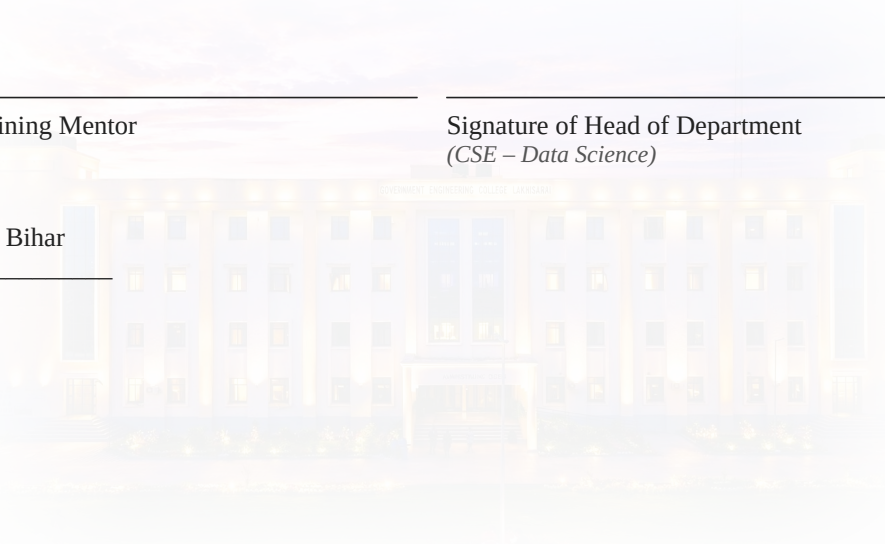
The work recorded in this report is found to be satisfactory and is submitted in partial fulfilment of the requirements for the award of the internship completion certificate.

Signature of Training Mentor
(Upeducative)

Signature of Head of Department
(CSE – Data Science)

Place: Lakhisarai, Bihar

Date: _____



DECLARATION

I, Isha, hereby declare that the work presented in this internship report titled “Student Result Prediction using Machine Learning” is an authentic record of my own work carried out during the internship period under the training partnership of Upeducative, under the guidance of Riya Kumari.

The dataset, code, results, and observations documented in this report were produced by me by following the training curriculum and executing the corresponding code myself. This report has not been submitted elsewhere for the award of any other certificate or qualification.

Isha

B.Tech (CSE – Data Science)
Government Engineering College, Lakhisarai



ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my training mentor, Riya Kumari, and the entire team at Upeducative for their guidance, patience, and structured teaching throughout this internship. The day-wise, hands-on format of the sessions made it possible to move from basic array operations to a working, evaluated machine learning model in a short span of time.

I am also thankful to the Department of Computer Science & Engineering (Data Science) and the faculty at Government Engineering College, Lakhisarai, for their continuous encouragement and for providing the opportunity to undertake this internship as part of my academic curriculum.

Finally, I would like to thank my family and peers for their constant support and encouragement during this internship.

Isha



TABLE OF CONTENTS

Certificate.....	2
Declaration.....	3
Acknowledgement.....	4
Chapter 1. Introduction.....	6
1.1 About the Internship.....	6
1.2 Objectives of the Internship.....	6
1.3 Tools and Technologies Used.....	6
Chapter 2. Foundations: Machine Learning and NumPy.....	9
Chapter 3. Introduction to Pandas.....	14
Chapter 4. The Dataset and Data Cleaning.....	17
Chapter 5. Exploratory Data Analysis.....	23
Chapter 6. Feature Scaling and Train/Test Split.....	28
Chapter 7. Model Building.....	31
Chapter 8. Model Evaluation.....	34
Chapter 9. Results and Discussion.....	37
Chapter 10. Challenges Faced.....	39
Chapter 11. Learning Outcomes.....	40
Chapter 12. Conclusion and Future Scope.....	41
References.....	43
Appendix — Complete Code Listing.....	44

CHAPTER 1

INTRODUCTION

1.1 About the Internship

This report documents the work carried out during an internship training programme conducted by Upeducative, focused on building a complete, end-to-end machine learning workflow using Python. The training was organised as a sequence of structured sessions, each building directly on the previous one — starting from numerical computing fundamentals and progressing through data handling, cleaning, exploration, model building, and evaluation.

The internship used a single, consistent project throughout — Student Result Prediction — so that every tool and technique learned could be applied immediately to real, hands-on work rather than isolated exercises. This report follows that same structure, chapter by chapter.



1.2 Objectives of the Internship

- To build a working understanding of NumPy for numerical computing and Pandas for structured data handling.
- To learn the practical steps of data cleaning — handling missing values, correcting data types, and removing duplicate records.
- To perform exploratory data analysis (EDA) in order to identify which features in a dataset are meaningfully related to an outcome of interest.
- To understand and apply feature scaling and a proper train/test split, including the importance of stratified sampling.
- To train, evaluate, and interpret a classification model — specifically Logistic Regression — using appropriate metrics for an imbalanced dataset.
- To recognise and avoid data leakage, identified during the training as the single most important discipline in building a trustworthy model.



1.3 Tools and Technologies Used

Tool / Library	Purpose in this Project
Python 3	Core programming language used throughout the internship
Google Colaboratory	Cloud notebook environment used for all hands-on coding sessions
NumPy	Numerical computing — arrays, reshaping, aggregation, Boolean masking
Pandas	Loading, cleaning, exploring, and transforming the tabular dataset
Matplotlib	Data visualisation — bar charts, histograms, boxplots, confusion matrix
scikit-learn	Train/test splitting, feature scaling, model training and evaluation



CHAPTER 2

FOUNDATIONS: MACHINE LEARNING AND NUMPY

2.1 What is Machine Learning?

Machine Learning is an approach to teaching computers to learn patterns from data, rather than depending on a human to write every rule by hand. In traditional programming, a set of Rules is combined with Data to produce an Output. In Machine Learning, the process is reversed: Data together with Expected Output is used to arrive at a Learned Pattern, referred to as a model.

Illustration: Shown 50 labelled pictures of cats and dogs, a child eventually learns to tell them apart without ever being given an explicit rule about whiskers or ear shape. This is the same principle by which an ML model learns from labelled data.

Three broad categories of Machine Learning were covered during the training:

Type	How It Learns	Example
Supervised Learning	From labelled data (input paired with the correct answer)	Predicting a house price from its area and location
Unsupervised Learning	By finding hidden structure in unlabelled data	Grouping customers by shopping behaviour
Reinforcement Learning	Through trial, error, and reward signals	A game-playing agent that improves as it scores points

The project undertaken in this internship, predicting Pass/Fail from student performance data, is an example of Supervised Learning, since each student record includes a known, correct outcome.

2.2 NumPy: The Foundation of Numerical Computing

NumPy (Numerical Python) is the library on which nearly every data-science and machine-learning tool in Python is built. Compared to a plain Python list, a NumPy array performs mathematical operations far faster, requires no explicit loop, and naturally supports one-, two-, and higher-dimensional data.

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])

arr.ndim      # 2      -- number of dimensions
arr.shape     # (2, 3) -- size along each dimension
arr.size      # 6      -- total number of elements
arr.dtype     # int64  -- data type of the elements
```



2.3 Indexing, Slicing, and Vectorised Operations

NumPy indexing begins at zero, and slicing follows the convention `arr[start:end]`, where the starting position is included and the ending position is excluded. Arithmetic operations on arrays are described as vectorised — a single operation, such as addition or multiplication, is applied to every element at once, without an explicit loop.

```
arr = np.array([10, 20, 30, 40, 50])
arr[0]      # 10   (first element)
arr[-1]     # 50   (last element)
arr[1:4]    # [20 30 40]

a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
a + b       # [11 22 33] -- vectorised, no loop needed
```



2.4 Reshaping and the Axis Concept

The `reshape()` function changes the shape of an array without altering its underlying data, provided the total number of elements stays the same. In a two-dimensional array, the `axis` parameter specifies the direction of an operation: `axis=0` operates down each column, while `axis=1` operates across each row.

```
arr = np.arange(1, 13).reshape(3, 4) # 12 elements -> 3 rows x 4 cols  
  
marks = np.array([[85,78,92],[67,88,74],[90,95,88]])  
marks.mean(axis=0) # [80.7 87.0 84.7] -- average per column (subject)  
marks.mean(axis=1) # [85.0 76.3 91.0] -- average per row (student)
```



2.5 Boolean Masking

Comparing an array against a value produces a Boolean array of the same shape, with True or False at every position. Using this Boolean array inside square brackets filters the original array, keeping only the elements where the condition is True. This single pattern, `arr[condition]`, underlies almost all data filtering in both NumPy and Pandas.

```
arr = np.array([4, 9, 2, 10, 6])
arr[arr > 5]          # [9 10 6]
arr[(arr > 3) & (arr < 9)] # [4 6]  -- AND
```



CHAPTER 3

INTRODUCTION TO PANDAS

3.1 Series and DataFrame

Pandas is built directly on top of NumPy, adding labels, named columns, and support for mixed data types. A Series is a single labelled column of data, while a DataFrame is a full table made up of multiple Series that share a common index — conceptually similar to a spreadsheet or a SQL table.

```
import pandas as pd

marks = pd.Series([85, 78, 92, 67], name="Math")
marks.mean()      # 80.5

df = pd.DataFrame({
    "Name": ["Aman", "Riya", "Karan"],
    "Math": [85, 67, 90],
})
```



3.2 Reading and Exploring a Dataset

Real datasets typically arrive as CSV files, which Pandas loads with a single call to `read_csv()`. Once loaded, a small set of commands is used repeatedly to understand the structure of any new dataset.

Command	Purpose
<code>df.head() / df.tail()</code>	Preview the first / last rows of the table
<code>df.shape</code>	Number of rows and columns
<code>df.columns</code>	List of all column names
<code>df.info()</code>	Column names, data types, and non-null counts
<code>df.describe()</code>	Mean, minimum, maximum, and other statistics for numeric columns



3.3 Selecting, Filtering, and Sorting

A single column selected with `df["col"]` returns a Series, while a list of columns selected with `df[["a","b"]]` returns a DataFrame. Rows are selected either by label, using `loc[]`, or by numeric position, using `iloc[]` — with the important distinction that label-based slicing with `loc` includes the final row, while position-based slicing with `iloc` excludes it. Rows can be filtered using the same Boolean-masking pattern used in NumPy, and sorted using `sort_values()`.

```
df["Math"]           # Series (single column)
df.loc[0:2]          # rows labelled 0,1,2 -- end included
df.iloc[0:2]         # rows at positions 0,1 -- end excluded
df[df["Math"] > 80]  # filter rows by a condition
df.sort_values("Math", ascending=False) # highest first
```



CHAPTER 4

THE DATASET AND DATA CLEANING

4.1 Dataset Description

The dataset used throughout this project, `student_performance_large.csv`, contains records for 300 students across 8 columns. The objective is to predict whether a student will Pass or Fail based on measurable study-related habits.

Column	Description
Student_ID	A unique identifier for each student (not used as a feature)
Hours_Studied	Average number of hours studied per day
Attendance	Attendance percentage
Previous_Score	Marks obtained in the previous semester (%)
Assignments_Completed	Number of assignments completed, out of 10
Sleep_Hours	Average number of hours slept per day
Final_Marks	Final examination marks — excluded from the feature set (Section 4.5)
Result	Pass / Fail — the target variable to be predicted

4.2 Loading the Dataset

The dataset was uploaded to Google Colaboratory and loaded using Pandas. Checking the shape of the DataFrame immediately after loading is a routine habit that confirms the full file was read without truncation.



```
Google Colab — Data Loading

1 import pandas as pd
2 import numpy as np
3
4 df = pd.read_csv("student_performance_large.csv")
5 df.shape

OUTPUT
(300, 8)
```

Figure 4.1 — Loading the dataset and confirming its shape



4.3 Identifying Missing Values

Running `isnull().sum()` across the DataFrame revealed 20 missing values in total — 4 missing entries in each of the 5 numeric feature columns. The `Student_ID`, `Final_Marks`, and `Result` columns had no missing values.

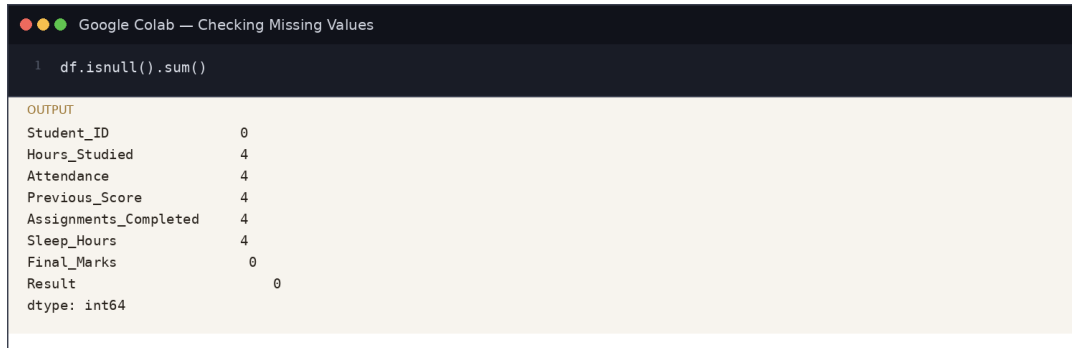


Figure 4.2 — Missing-value count by column



4.4 Handling Missing Values, Data Types, and Duplicates

Since fewer than 2% of values were missing in any single column, filling them with the column mean was judged a safe and standard approach. This step, however, converted `Assignments_Completed` — a count that should only take whole-number values — into a decimal column, so it was rounded and converted back to an integer type immediately afterward. The dataset was then checked for duplicate rows.



```
Google Colab — Cleaning the Dataset

1 df = df.fillna(df.mean(numeric_only=True))
2 df["Assignments_Completed"] = df["Assignments_Completed"]\
3     .round().astype(int)
4 df.duplicated().sum()
5 df.isnull().sum().sum()

OUTPUT
0      # no duplicate rows found
0      # zero missing values remain
```

Figure 4.3 — Filling missing values, correcting the data type, and checking for duplicates

Observation: No duplicate rows were found in this dataset; the check was still performed, since a clean-looking dataset should never be assumed duplicate-free without verification.



4.5 Avoiding Data Leakage

The Result column used as the prediction target was itself derived directly from Final_Marks (a student passes if Final_Marks is at least 45). Including Final_Marks as an input feature would therefore allow the model to simply read the answer rather than learn any genuine relationship — a problem known as data leakage. This was treated as the single most important rule of the entire project: Final_Marks is present in the dataset for reference, but is deliberately excluded from the feature set used for training.

```
# INCORRECT -- Final_Marks leaks the answer directly
X = df[["Hours_Studied", "Final_Marks"]]

# CORRECT -- Final_Marks excluded from the feature set
feature_cols = ["Hours_Studied", "Attendance", "Previous_Score",
                "Assignments_Completed", "Sleep_Hours"]
X = df[feature_cols]
y = df["Result"]
```



4.6 Saving the Cleaned Dataset

Once cleaning was complete, the DataFrame was saved as `student_performance_clean.csv`, so that every subsequent stage of the project could begin directly from this file without repeating the cleaning steps.



CHAPTER 5

EXPLORATORY DATA ANALYSIS

5.1 Purpose of This Stage

Before any model is trained, it is necessary to understand which features in the cleaned dataset actually relate to the target outcome. This stage does not involve training a model; its purpose is solely to determine what is worth feeding into one.



5.2 Class Balance

The first and most important check in a classification problem is whether the target classes are evenly represented. Counting the values in the Result column showed a clear imbalance.



Figure 5.1 — Class distribution of the Result column

Why this matters: A model that predicted Pass for every single student would still be correct 83% of the time without learning anything at all. This is why the model in Chapter 8 is judged using more than plain accuracy, and why the train/test split in Chapter 6 was stratified to preserve this 83/17 ratio in both sets.



5.3 Comparing Pass and Fail Students

Grouping the dataset by Result and computing the mean of each feature provided a direct, numeric comparison between the two groups.

Google Colab — Comparing Pass vs Fail					
<pre>1 feature_cols = ["Hours_Studied", "Attendance", "Previous_Score", 2 "Assignments_Completed", "Sleep_Hours"] 3 df.groupby("Result")[feature_cols].mean().round(2)</pre>					
OUTPUT					
	Hours_Studied	Attendance	Prev_Score	Assign	Sleep
Fail	1.93	63.64	55.48	3.55	6.07
Pass	5.50	71.48	66.61	5.37	6.56

Figure 5.2 — Average feature values for Pass vs. Fail

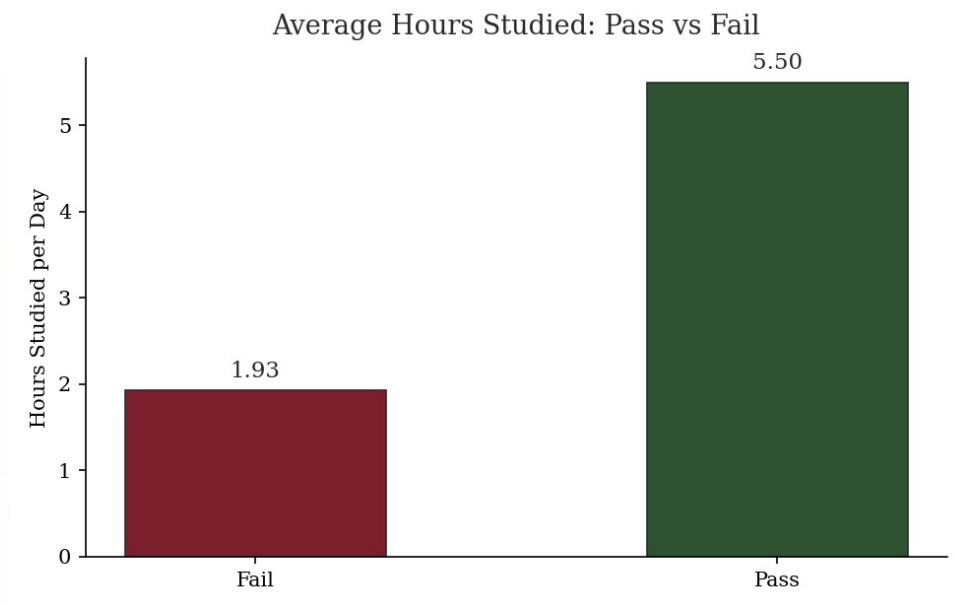


Figure 5.3 — Average daily study hours, Pass vs. Fail

The gap in Hours_Studied was by far the largest — students who passed studied close to three times as many hours per day, on average, as those who failed. Previous_Score showed the next largest gap, while Attendance, Assignments_Completed, and Sleep_Hours differed by smaller margins.

5.4 Distribution Analysis

Averages alone can hide the shape of the underlying data, so the distribution of individual features was also examined directly, using a histogram and a boxplot.

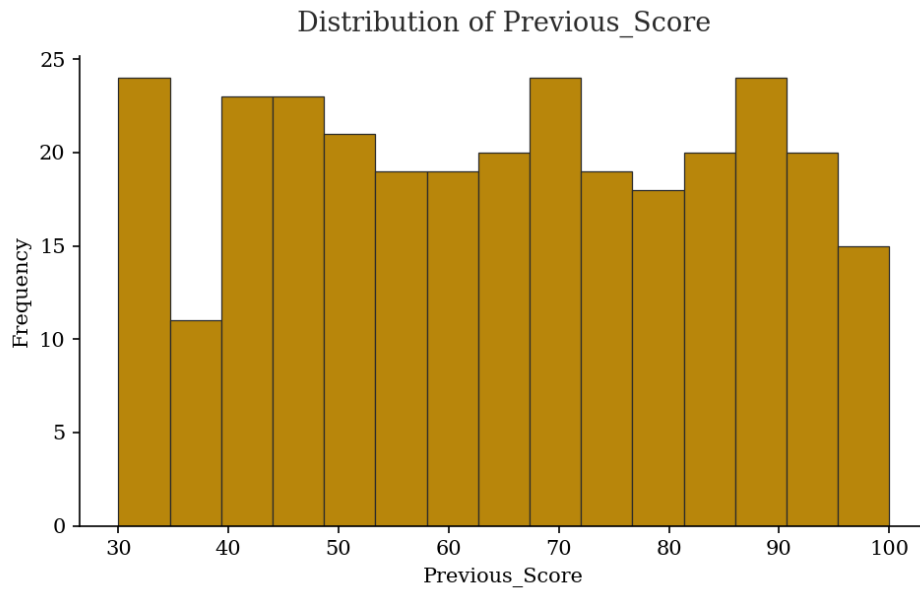


Figure 5.4 — Distribution of Previous_Score across all students

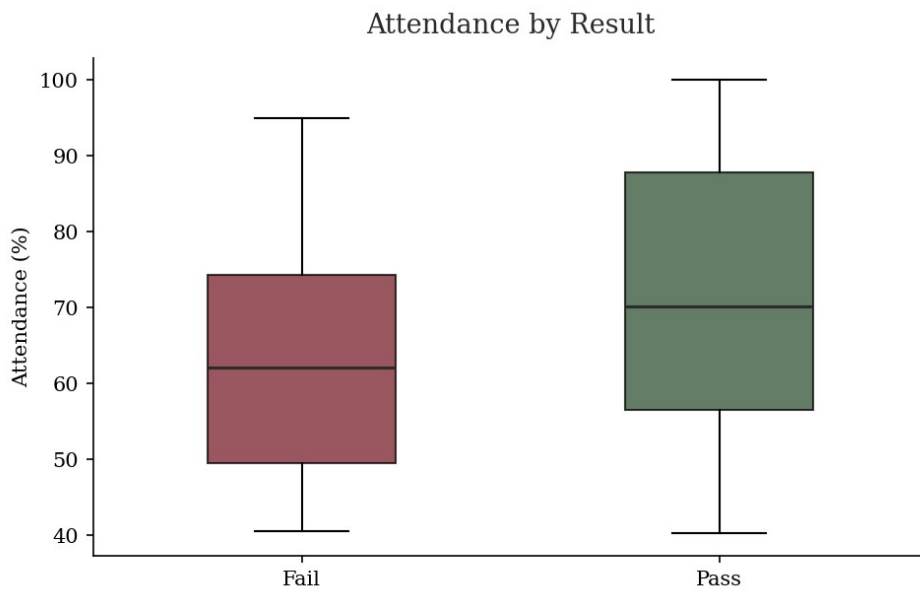


Figure 5.5 — Attendance spread, grouped by Result

The boxplot shows that Pass students have both a higher median attendance and a wider spread reaching up to 100%, but the interquartile ranges of the two groups overlap considerably — indicating that Attendance alone would not cleanly separate the two classes.

5.5 Correlation Analysis

To confirm the pattern observed from averages and distributions, the Result column was converted to a numeric form (Pass = 1, Fail = 0), and the correlation of each feature with this numeric target was computed.



Figure 5.6 — Correlation of each feature with the Result

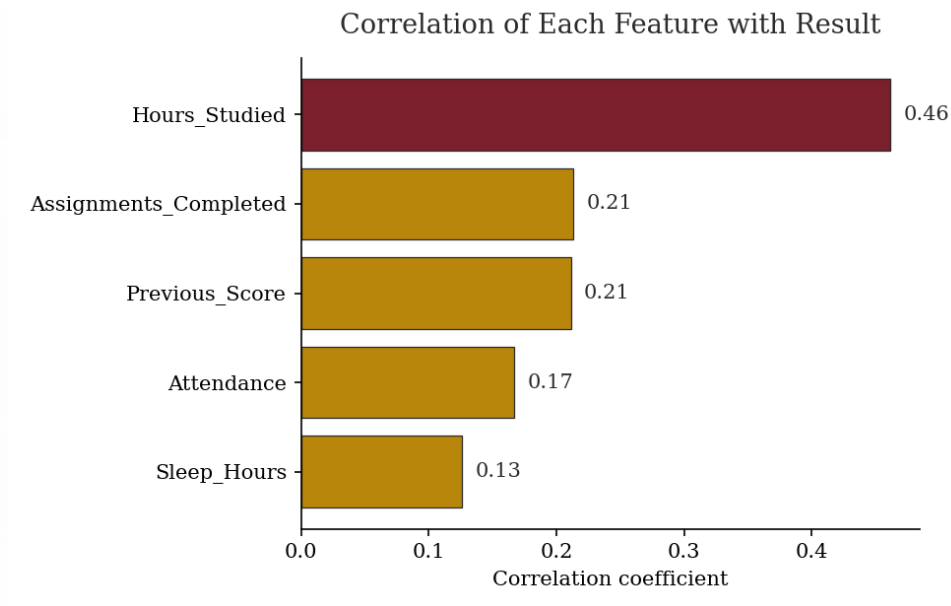


Figure 5.7 — Correlation strength, visualised

Hours_Studied showed by far the strongest correlation with passing (0.46), roughly double that of the next two features, Assignments_Completed and Previous_Score (0.21 each). Attendance (0.17) and Sleep_Hours (0.13) showed weaker, though still positive, relationships with the outcome. This did not lead to any feature being discarded; rather, it established Hours_Studied as the strongest individual signal to watch once a model was trained.

CHAPTER 6

FEATURE SCALING AND TRAIN/TEST SPLIT

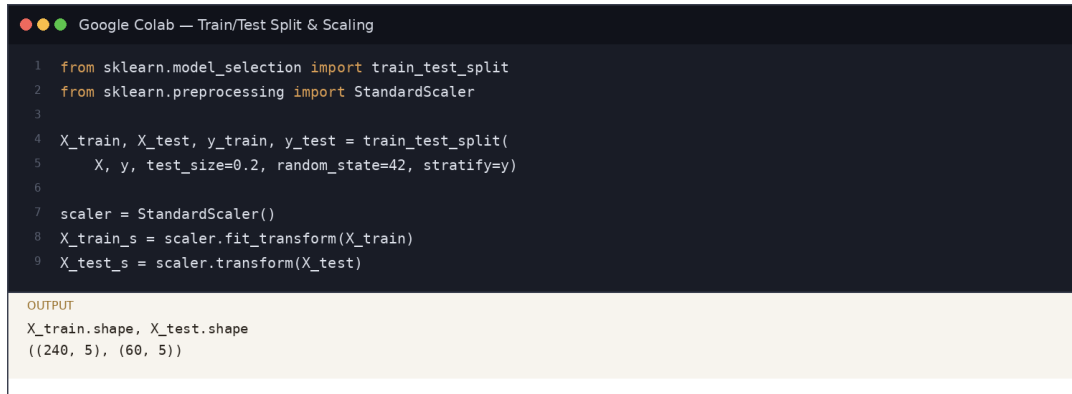
6.1 Splitting the Data

Before training any model, the cleaned dataset was divided into a training set and a test set, using scikit-learn's `train_test_split()`. Eighty percent of the 300 records (240 rows) were reserved for training, and the remaining twenty percent (60 rows) were held back purely for evaluation, never seen by the model during training.



6.2 Stratified Sampling

Because the dataset is imbalanced (83% Pass, 17% Fail), a plain random split risks distorting this ratio in the smaller test set — with only 51 Fail records in the entire dataset, an unlucky split could easily under- or over-represent them. Passing `stratify=y` to `train_test_split()` forces both the training and test sets to preserve the original class ratio as closely as possible.



```

1 from sklearn.model_selection import train_test_split
2 from sklearn.preprocessing import StandardScaler
3
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, test_size=0.2, random_state=42, stratify=y)
6
7 scaler = StandardScaler()
8 X_train_s = scaler.fit_transform(X_train)
9 X_test_s = scaler.transform(X_test)

```

OUTPUT

```

X_train.shape, X_test.shape
((240, 5), (60, 5))

```

Figure 6.1 — Stratified train/test split and feature scaling

Set	Rows	Pass	Fail	% Pass
Training set	240	199	41	82.9%
Test set	60	50	10	83.3%

Both the training and test sets ended up within a single percentage point of the original 83.0% Pass ratio in the full dataset, confirming that the stratified split behaved as intended.

6.3 Feature Scaling with StandardScaler

The five numeric features used by this model do not share a common scale — Attendance ranges from roughly 40 to 100, while Hours_Studied ranges from 0 to about 10. Left unscaled, a model can be misled into treating a numerically larger feature as more important, purely because of its range rather than its actual relationship with the outcome. StandardScaler was used to rescale every feature to a mean of 0 and a standard deviation of 1.

A second form of data leakage: The scaler was fitted only on the training data, using `fit_transform(X_train)`, and then applied to the test data using `transform(X_test)` without being re-fitted. Fitting the scaler on the full dataset before splitting would let information from the test set quietly influence the training process — the same underlying mistake as the `Final_Marks` leakage described in Chapter 4, applied here to scaling instead of features.



CHAPTER 7

MODEL BUILDING

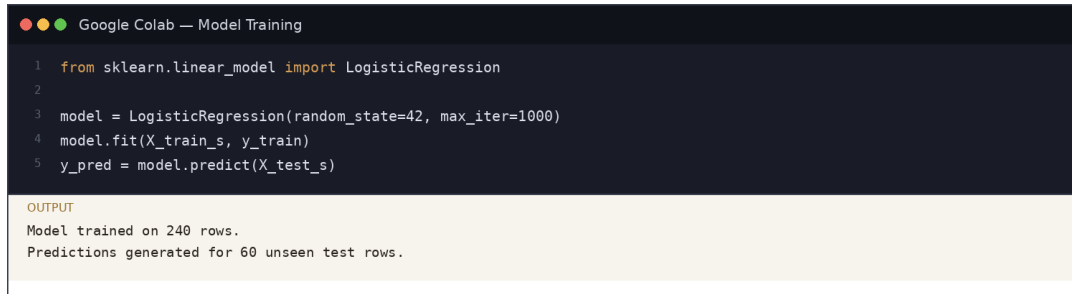
7.1 Choice of Algorithm

For a two-outcome (Pass/Fail) prediction problem, Logistic Regression was selected as the model to train. Rather than only producing a hard Pass/Fail label, it estimates the underlying probability of each outcome, trains quickly, and produces coefficients that are straightforward to interpret — making it a strong, explainable first model to evaluate before considering more complex alternatives.



7.2 Training the Model

The model was trained by calling `fit()` on the scaled training features and their corresponding labels. Training used only the 240 rows in the training set; the 60-row test set remained completely untouched at this stage.



```
Google Colab — Model Training

1 from sklearn.linear_model import LogisticRegression
2
3 model = LogisticRegression(random_state=42, max_iter=1000)
4 model.fit(X_train_s, y_train)
5 y_pred = model.predict(X_test_s)

OUTPUT
Model trained on 240 rows.
Predictions generated for 60 unseen test rows.
```

Figure 7.1 — Fitting the Logistic Regression model



7.3 Interpreting the Model's Coefficients

Once trained, the model's coefficients indicate how strongly each feature pulls a prediction toward Pass, holding the other features constant.

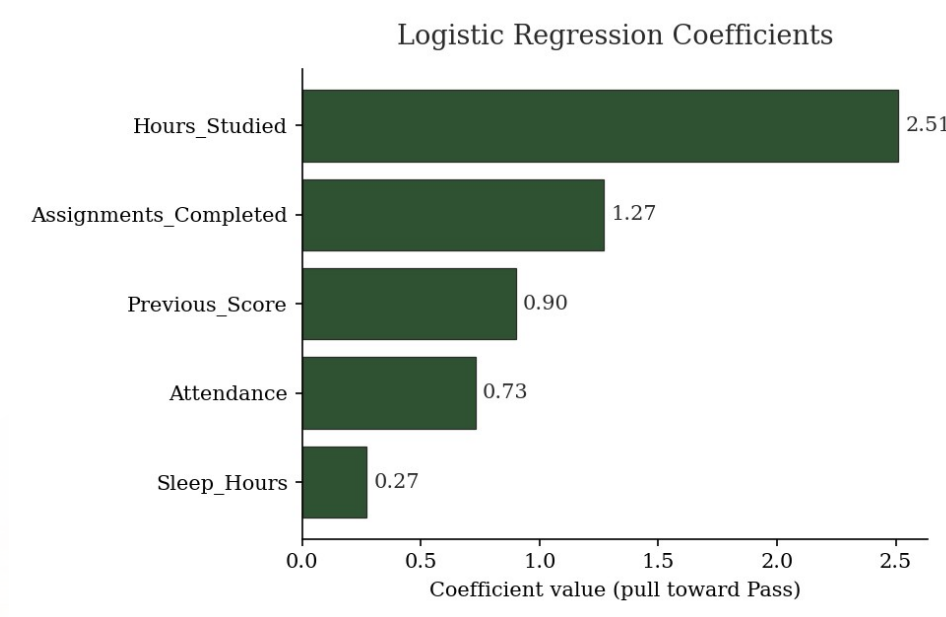


Figure 7.2 — Logistic Regression coefficients by feature

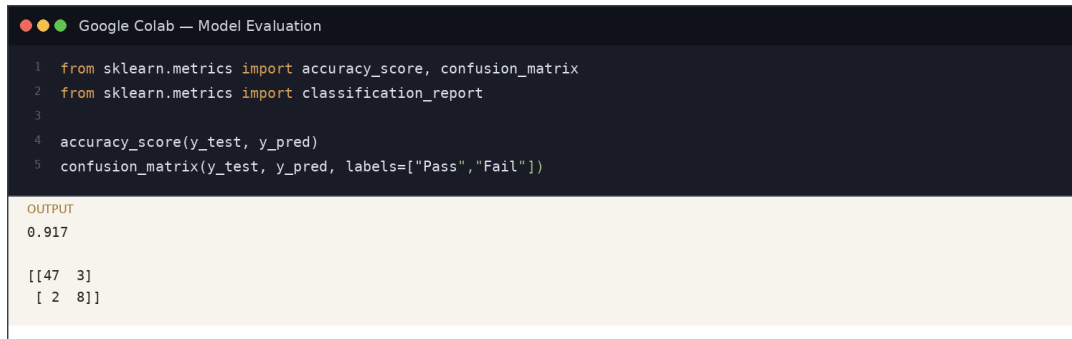
Hours_Studied received the largest coefficient by a wide margin, followed by Assignments_Completed, Previous_Score, Attendance, and Sleep_Hours — a ranking that closely mirrors the correlation values found during exploratory analysis in Chapter 5.

CHAPTER 8

MODEL EVALUATION

8.1 Making Predictions and Checking Accuracy

Using the trained model, predictions were generated for the 60 rows in the test set — data the model had never encountered during training. Comparing these predictions against the true Result values gave an overall accuracy figure.



```
Google Colab — Model Evaluation

1 from sklearn.metrics import accuracy_score, confusion_matrix
2 from sklearn.metrics import classification_report
3
4 accuracy_score(y_test, y_pred)
5 confusion_matrix(y_test, y_pred, labels=["Pass", "Fail"])

OUTPUT
0.917

[[47  3]
 [ 2  8]]
```

Figure 8.1 — Accuracy and confusion matrix on the test set

The model classified 91.7% of the 60 test students correctly on its first evaluation.

8.2 The Confusion Matrix

A confusion matrix breaks a classifier's predictions down by the type of outcome, rather than a single aggregate number, distinguishing correct predictions from the two different kinds of mistake a classifier can make.

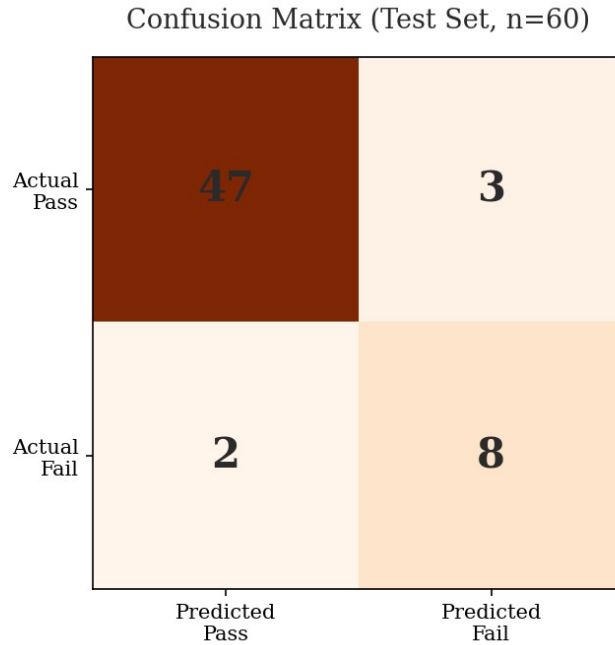


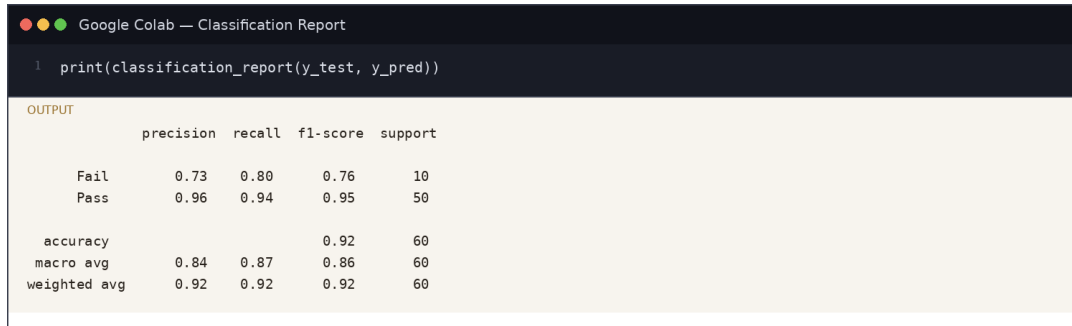
Figure 8.2 — Confusion matrix for the test set ($n = 60$)

	Predicted: Yes	Predicted: No
Actual: Yes	True Positive (TP)	False Positive (FP)
Actual: No	False Negative (FN)	True Negative (TN)

Note on layout: This exact row/column arrangement of the confusion matrix is the one used in the training material and is retained here as taught, even though some textbooks present the four quadrants in a different order.

8.3 Precision, Recall, and F1-Score

Because the dataset is imbalanced, accuracy alone can be misleading — a model that always predicted Pass would already score 83% without learning anything. The classification report therefore breaks performance down per class.



```

print(classification_report(y_test, y_pred))

```

	precision	recall	f1-score	support
Fail	0.73	0.80	0.76	10
Pass	0.96	0.94	0.95	50
accuracy			0.92	60
macro avg	0.84	0.87	0.86	60
weighted avg	0.92	0.92	0.92	60

Figure 8.3 — Precision, recall, and F1-score for each class

Metric	Meaning
Accuracy	Proportion of all predictions that were correct
Precision	Of all students predicted as a class, how many actually belonged to it
Recall	Of all students who actually belonged to a class, how many were correctly identified
F1-Score	The balance between precision and recall, combined into a single figure

Why Fail recall matters most here: In a result-prediction system, a missed Fail (a struggling student wrongly predicted to Pass) is the more costly mistake, since that student never gets flagged for support. The model's recall for the Fail class was 0.80, meaning it correctly identified 8 out of every 10 at-risk students in the test set — the single metric most worth improving in any future iteration of this model.

CHAPTER 9

RESULTS AND DISCUSSION

9.1 Summary of Findings

- Hours_Studied was the single strongest predictor of a student's Result, both by correlation (0.46) and by the trained model's own coefficient — students who passed studied nearly three times as many hours per day, on average, as those who failed.
- The dataset carried a meaningful class imbalance (83% Pass, 17% Fail), which shaped every later decision — from stratifying the train/test split to choosing which evaluation metric to prioritise.
- The trained Logistic Regression model reached 91.7% accuracy on unseen test data, with a Fail-class recall of 0.80, correctly flagging 8 out of 10 genuinely at-risk students.
- Excluding Final_Marks from the feature set, and fitting the scaler on training data only, were both identified as essential steps to avoid data leakage and obtain a trustworthy, generalisable model.



9.2 Limitations

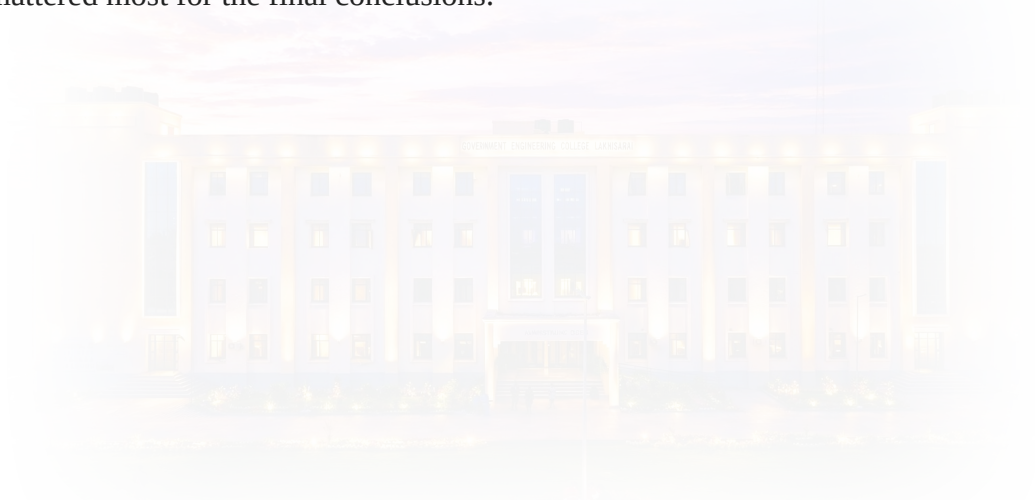
The dataset used in this project was limited to 300 records and five behavioural features. A larger and more diverse dataset, along with additional features such as prior subject-wise performance or engagement metrics, would likely improve both the reliability of the exploratory findings and the model's predictive accuracy — particularly for the minority Fail class, where only 51 records were available in total.



CHAPTER 10

CHALLENGES FACED

- Understanding why data leakage through Final_Marks would produce a misleadingly perfect-looking model required rethinking the feature set carefully rather than simply including every available column.
- Recognising that a second, quieter form of leakage could occur through the feature scaler itself — and that `scaler.fit()` must only ever see the training data — was a subtler lesson that took closer attention than the first leakage rule.
- Working with an imbalanced dataset meant that accuracy alone was not a sufficient way to judge the model, and required learning to read a confusion matrix and a classification report correctly.
- Translating exploratory observations (averages, distributions, correlations) into a clear narrative for this report, rather than simply listing numbers, required deciding which findings mattered most for the final conclusions.



CHAPTER 11

LEARNING OUTCOMES

This internship provided practical, hands-on experience with the complete lifecycle of a supervised machine learning project — from raw, uncleaned data through to a trained and evaluated model. The specific technical skills developed include:

- Working confidently with NumPy arrays, including reshaping, axis-based aggregation, and Boolean masking.
- Using Pandas to load, explore, clean, filter, and transform tabular data.
- Applying a systematic data-cleaning routine — missing values, data types, and duplicates — to any new dataset.
- Conducting exploratory data analysis using grouped statistics, visualisations, and correlation to decide which features are worth modelling.
- Understanding, and correctly implementing, a stratified train/test split and feature scaling using scikit-learn.
- Training a Logistic Regression classifier and evaluating it using accuracy, a confusion matrix, precision, recall, and F1-score.
- Most importantly, developing the habit of questioning whether a feature or a data-preparation step could leak information from outside the training set — a discipline that applies to any machine learning project, not only this one.

CHAPTER 12

CONCLUSION AND FUTURE SCOPE

12.1 Conclusion

This internship successfully delivered a complete, working machine learning pipeline for predicting student academic outcomes, built entirely from first principles across a structured, day-wise curriculum. Starting from raw NumPy arrays and progressing through Pandas-based data handling, systematic data cleaning, exploratory analysis, and finally model training and evaluation, the project demonstrates that Hours_Studied, Previous_Score, and Assignments_Completed are the features most strongly associated with a student passing, and that a simple, interpretable Logistic Regression model can predict this outcome with 91.7% accuracy on unseen data.



12.2 Future Scope

- Collecting a larger dataset, particularly more Fail-class records, to improve the reliability of minority-class predictions.
- Experimenting with additional classification algorithms, such as Decision Trees or Random Forests, and comparing their performance against the Logistic Regression baseline established in this report.
- Applying techniques for imbalanced classification, such as class weighting or resampling, to further improve Fail-class recall.
- Deploying the trained model behind a simple interface so that Attendance, Hours_Studied, and similar inputs for a new student could be entered directly to obtain a prediction.



REFERENCES

- [1] NumPy Developers. NumPy Documentation. <https://numpy.org/doc/>
- [2] The Pandas Development Team. Pandas Documentation. <https://pandas.pydata.org/docs/>
- [3] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 2011. <https://scikit-learn.org/stable/>
- [4] Hunter, J. D. Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 2007. <https://matplotlib.org/>
- [5] Upeducative Training Curriculum — Student Result Prediction Project, Internship Sessions, 2026.



APPENDIX — COMPLETE CODE LISTING

The complete, consolidated pipeline used across this project is reproduced below, from raw data through to a trained and evaluated model.

```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. Load
df = pd.read_csv("student_performance_large.csv")

# 2. Clean
df = df.fillna(df.mean(numeric_only=True))
df["Assignments_Completed"] = df["Assignments_Completed"].round().astype(int)
df = df.drop_duplicates().reset_index(drop=True)

# 3. Features and target (Final_Marks excluded -- data leakage)
feature_cols = ["Hours_Studied", "Attendance", "Previous_Score",
                "Assignments_Completed", "Sleep_Hours"]
X = df[feature_cols]
y = df["Result"]
df.to_csv("student_performance_clean.csv", index=False)

# 4. EDA
df["Result"].value_counts(normalize=True) * 100
df.groupby("Result")[feature_cols].mean().round(2)
df["Result_num"] = (df["Result"] == "Pass").astype(int)
df[feature_cols + ["Result_num"]].corr()["Result_num"]

# 5. Split and scale
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# 6. Train and evaluate
model = LogisticRegression(random_state=42, max_iter=1000)
model.fit(X_train_s, y_train)
y_pred = model.predict(X_test_s)
accuracy_score(y_test, y_pred) # 0.917
confusion_matrix(y_test, y_pred, labels=["Pass", "Fail"])
print(classification_report(y_test, y_pred))
```